

Advanced DAX

Data Analysis Expressions

Complete Learning Guide for Business Intelligence Professionals

Topics Covered in This Chapter

- ▶ Time Intelligence Functions — DATESYTD, DATESMTD, SAMEPERIODLASTYEAR
- ▶ Filter Modifiers — ALL and ALLEXCEPT
- ▶ Variables in DAX — Cleaner, Faster Formulas
- ▶ Conditional Logic — IF and SWITCH
- ▶ Ranking Functions — RANKX and Related Functions
- ▶ Performance Basics — Writing Efficient DAX
- ▶ Practice Exercise — End-to-End Application

Introduction to Advanced DAX

In the previous module, you were introduced to the basics of DAX — what it is, how it works, and how to write simple calculated columns and measures. In this chapter, we go further.

Advanced DAX is where business intelligence truly comes to life. Instead of just calculating totals, you will learn to analyse data over time, remove filters, simplify complex logic, and rank items — all using DAX formulas inside Power BI or Excel Power Pivot.

This chapter has been designed for learners who have a foundational understanding of DAX and are ready to apply it to real business scenarios. Each topic builds on the previous one, and every concept is introduced step-by-step with clear examples.



What You Will Learn in This Chapter

How to calculate running totals, monthly figures, and year-over-year comparisons using Time Intelligence functions.

How to remove or override filters on your data using ALL and ALLEXCEPT.

How to write cleaner, more readable, and more efficient formulas using Variables.

How to apply conditional logic using IF and SWITCH to make decisions inside measures.

How to rank items (products, salespeople, regions) dynamically using RANKX.

How to write DAX formulas that load and calculate faster — Performance Basics.

What You Should Already Know

Before starting this chapter, make sure you are comfortable with the following concepts:

Concept	Why It Is Needed in This Chapter
Calculated Columns and Measures	Advanced DAX builds on both of these — most formulas you will write are measures.
CALCULATE function basics	Many advanced functions are used inside CALCULATE to modify filter context.
Date table / calendar table	Time Intelligence functions require a properly structured date table in your model.
Basic SUM, COUNT, AVERAGE	These will serve as the base calculations inside more complex formulas.
Filter context and row context	Understanding how DAX evaluates context is key to all advanced functions.

Section 1: Time Intelligence Functions

Time Intelligence is one of the most valuable capabilities in DAX. It allows you to perform calculations across different time periods — for example, comparing this month's revenue to last month's, or showing a running total of sales from the beginning of the year.

These functions do not work in isolation. They require a Date table — a dedicated table in your data model that contains one row for every date in your analysis range, with no gaps. This is a requirement, not a suggestion.

1.1 What Is a Date Table and Why Is It Required?

A Date table (also called a Calendar table) is a special table in your data model. It contains one row per day, and its Date column must be marked as a Date Table in Power BI.

A basic Date table must have at minimum:

- A Date column — containing every single date in your data range (no missing dates).
- A Year column — the four-digit year (e.g., 2023, 2024).
- A Month Number column — 1 to 12.
- A Month Name column — January, February, etc.
- A Quarter column — Q1, Q2, Q3, Q4.

Pro Tip

Always build or import a Date table separately — do not rely on auto-generated date hierarchies in Power BI.

Your Date table must cover the full range of dates in your data. If your sales data goes from 01 Jan 2020 to 31 Dec 2024, your Date table must cover that entire range.

Mark the table as a Date Table in Power BI: go to Model view, right-click the Date table, and select Mark as Date Table.

1.2 Sample Data — Monthly Sales

Throughout Section 1, we will use the following dataset. This represents monthly sales data for a company across three years.

Date	Month	Year	Region	Sales Amount (₹)
01-Jan-2023	January	2023	North	1,20,000
01-Feb-2023	February	2023	North	95,000
01-Mar-2023	March	2023	North	1,40,000
01-Apr-2023	April	2023	North	1,10,000
01-Jan-2024	January	2024	North	1,35,000
01-Feb-2024	February	2024	North	1,05,000

Date	Month	Year	Region	Sales Amount (₹)
01-Mar-2024	March	2024	North	1,60,000
... (data continues)	...	...	...	...

Figure 1.2 — Sample Monthly Sales Data

1.3 Year-to-Date (YTD) — DATESYTD

YTD — Year-to-Date

A cumulative running total from the first day of the year up to the current date. For example, if today is 15 March, YTD Sales = January Sales + February Sales + March Sales (up to 15th).

What Does YTD Tell You?

YTD (Year-to-Date) helps you understand how performance has accumulated over the course of the year. It answers questions like:

- "How much total revenue have we generated this year so far?"
- "Are we on track to meet our annual target?"
- "How does our YTD performance compare to the same point last year?"

The TOTALYTD Function

DAX provides a simple function called TOTALYTD that calculates a year-to-date total. Here is the syntax:

DAX

```
TOTALYTD( <Expression>, <Dates> [, <Filter>] [, <YearEndDate>] )
```

-- Parameters:

-- Expression : The value you want to sum or calculate (e.g., SUM of Sales)

-- Dates : The date column from your Date table

-- Filter : Optional — an additional filter condition

-- YearEndDate : Optional — for fiscal years that do not end on 31 Dec

Figure 1.3 — TOTALYTD Syntax

Example 1 — YTD Sales (Calendar Year)

Suppose your Sales table has a column called [Sales Amount]. You want to create a measure that shows the total sales from 1st January up to the selected date.

DAX

YTD Sales =

```
TOTALYTD(
    SUM( Sales[Sales Amount] ),
    'Date'[Date]
```

```
)
```

Figure 1.4 — YTD Sales Measure

How to Read This Formula

Read this formula as: "Calculate the Sum of Sales Amount, accumulating from the beginning of the year up to the current date in the Date table."

 Step-by-Step Breakdown

- 1. SUM(Sales[Sales Amount]) — This is the base calculation. It adds up all values in the Sales Amount column.
- 2. 'Date'[Date] — This tells DAX to use the Date column from your Date table to determine the year-to-date range.
- 3. TOTALYTD wraps the SUM and automatically accumulates from 1st January to the current date in context.

Example 2 — YTD Sales for a Fiscal Year (April to March)

In many businesses — especially in India — the financial year runs from April 1st to March 31st. If your year ends on 31 March instead of 31 December, you must specify this using the YearEndDate parameter.

```
DAX
YTD Sales (Fiscal Year) =
TOTALYTD(
    SUM( Sales[Sales Amount] ),
    'Date'[Date],
    "31-03" -- Year End Date in MM-DD format
)
```

Figure 1.5 — YTD Sales for Fiscal Year (April–March)

 Note

The YearEndDate parameter must be written as a string in "MM-DD" format.
Example: 31st March = "03-31", 30th June = "06-30", 31st December = "12-31" (default).
Use this parameter only when your financial year does NOT end on 31st December.

Visualising YTD in a Table

When you add YTD Sales to a table visual in Power BI alongside your regular Sales measure, here is what you would typically see:

Month	Sales Amount (₹)	YTD Sales (₹)
January 2024	1,35,000	1,35,000
February 2024	1,05,000	2,40,000

Month	Sales Amount (₹)	YTD Sales (₹)
March 2024	1,60,000	4,00,000
April 2024	1,25,000	5,25,000
May 2024	1,45,000	6,70,000

Figure 1.6 — Monthly Sales vs. YTD Sales in a table visual

1.4 Month-to-Date (MTD) — DATESMTD

MTD — Month-to-Date

A cumulative total from the first day of the current month up to the selected date. For example, if today is 18 March, MTD Sales = Sales from 1st March to 18th March.

What Does MTD Tell You?

MTD is used when you want to track performance within a single month. It answers questions like:

- "How much have we sold so far this month?"
- "Are we on pace to hit our monthly target?"
- "Which day of the month are sales strongest?"

The TOTALMTD Function

DAX

```
TOTALMTD( <Expression>, <Dates> [, <Filter>] )
```

-- Expression : The value to calculate (e.g., SUM of Sales)

-- Dates : The date column from your Date table

-- Filter : Optional — additional filter to apply

Figure 1.8 — TOTALMTD Syntax

Example — MTD Sales

DAX

MTD Sales =

```
TOTALMTD(
    SUM( Sales[Sales Amount] ),
    'Date'[Date]
)
```

Figure 1.9 — MTD Sales Measure

YTD vs. MTD — When to Use Which?

Use YTD when you want to see the big picture — performance since the start of the year.

Use MTD when you want to monitor progress within a single month — ideal for monthly targets and short-term tracking.

Both are often shown together on executive dashboards alongside the base measure.

MTD vs. YTD — Side-by-Side Comparison

Date	Daily Sales (₹)	MTD Sales (₹)	YTD Sales (₹)
01-Mar-2024	32,000	32,000	3,72,000
02-Mar-2024	28,000	60,000	4,00,000
03-Mar-2024	41,000	1,01,000	4,41,000
04-Mar-2024	35,000	1,36,000	4,76,000
05-Mar-2024	29,000	1,65,000	5,05,000

Figure 1.10 — Daily, MTD, and YTD Sales compared

1.5 Same Period Last Year — SAMEPERIODLASTYEAR

SAMEPERIODLASTYEAR

Returns a set of dates that corresponds to the same period in the previous year. Use it to compare current performance against the same timeframe last year.

CLOUD & AI ACADEMY

Why Is Year-over-Year Comparison Important?

Comparing this year's numbers to last year's is one of the most common analyses in business. It helps you understand:

- Whether the business is growing, declining, or staying flat.
- Whether a good or bad month this year is unusual compared to the same month last year.
- Seasonality — whether high sales in a particular month are normal for that time of year.

Syntax

DAX

SAMEPERIODLASTYEAR(<Dates>)

-- Dates : The date column from your Date table

-- This function is used INSIDE CALCULATE:

Sales LY =

CALCULATE(

SUM(Sales[Sales Amount]),

SAMEPERIODLASTYEAR('Date'[Date])

```
)
```

Figure 1.11 — SAMEPERIODLASTYEAR Syntax and Example

Example 1 — Sales Last Year (Basic)

Create a measure that always shows the equivalent period from last year, regardless of what date or period is selected in a slicer.

```
DAX
Sales Last Year =
CALCULATE(
    SUM( Sales[Sales Amount] ),
    SAMEPERIODLASTYEAR( 'Date'[Date] )
)
```

Figure 1.12 — Sales Last Year Measure

Example 2 — Year-over-Year Growth Percentage

Once you have Sales Last Year, you can calculate the percentage growth easily:

```
DAX
YoY Growth % =
VAR CurrentSales = SUM( Sales[Sales Amount] )
VAR LastYearSales =
    CALCULATE(
        SUM( Sales[Sales Amount] ),
        SAMEPERIODLASTYEAR( 'Date'[Date] )
    )
RETURN
    DIVIDE( CurrentSales - LastYearSales, LastYearSales, 0 )
```

Figure 1.13 — YoY Growth % Measure



Reading the Formula

1. VAR CurrentSales — stores the total sales for the current period.
2. VAR LastYearSales — stores the total sales for the same period last year.
3. DIVIDE — divides the difference by last year's sales to get the growth percentage.
4. The third argument in DIVIDE (0) is the alternate result — returned when division by zero would occur.

Sample Output — Monthly Comparison

Month	Sales 2024 (₹)	Sales 2023 (₹)	YoY Growth %
January	1,35,000	1,20,000	+12.5%
February	1,05,000	95,000	+10.5%
March	1,60,000	1,40,000	+14.3%
April	1,25,000	1,10,000	+13.6%

Figure 1.14 — YoY Sales Comparison Table

Time Intelligence — Quick Reference Summary

Function	What It Does	Common Use
TOTALYTD	Cumulative total from Jan 1st to current date	Annual performance tracking
TOTALMTD	Cumulative total from 1st of month to current date	Monthly progress monitoring
TOTALQTD	Cumulative total from 1st of quarter to current date	Quarterly reporting
SAMEPERIODLASTYEAR	Returns the same period from the previous year	Year-over-year comparison
DATEADD	Shifts a set of dates by a specified interval	Custom period comparisons

Figure 1.16 — Time Intelligence Functions Summary Table

 Important

Time Intelligence functions WILL NOT work unless your Date table is properly marked as a Date Table in Power BI.

There must be NO gaps in your date column — every date in your range must have a row.

Ensure your fact table (Sales, Orders, etc.) has a relationship with the Date table on the date column.

Section 2: Filter Modifiers — ALL and ALLEXCEPT

To understand ALL and ALLEXCEPT, you first need to understand the concept of Filter Context — because these functions exist specifically to modify or remove it.

2.1 What Is Filter Context? (A Quick Recap)

When you place a measure in a Power BI table or chart visual, DAX automatically evaluates it within the context of the row or visual you are in. This is called Filter Context.



Example: How Filter Context Works

Imagine a table visual in Power BI with three columns: Region, Product, Total Sales.

When DAX calculates Total Sales for row [North, Laptop], it automatically filters the Sales table to only include rows where Region = North AND Product = Laptop.

This automatic filtering is called Filter Context — it is applied by the visual and passed down to the DAX engine.

ALL removes this filter context, allowing you to calculate across ALL data, ignoring what the visual selected.

2.2 The ALL Function

ALL(Table or Column)

Removes all filters from a specified table or column. Returns all rows in the table regardless of any active filters in the report or visual.

Syntax

```
DAX
ALL( <TableNameOrColumnName> )

-- You can pass:
-- ALL( 'Sales' )      -- removes ALL filters from the entire Sales table
-- ALL( Sales[Region] ) -- removes filters only from the Region column
-- ALL( Sales[Region], Sales[Product] ) -- removes filters from both columns
```

Figure 2.1 — ALL Function Syntax

The Most Common Use: Calculating a Percentage of Total

The most practical application of ALL is calculating each item's percentage contribution to the overall total.

Scenario: You have a sales report by product. You want to add a column showing what percentage of total revenue each product contributes.

```

DAX

-- Step 1: Create the base measure
Total Sales =
SUM( Sales[Sales Amount] )

-- Step 2: Create the Overall Total using ALL
Overall Total Sales =
CALCULATE(
    SUM( Sales[Sales Amount] ),
    ALL( Sales )
)

-- Step 3: Calculate the Percentage
Sales % of Total =
DIVIDE(
    [Total Sales],
    [Overall Total Sales],
    0
)

```

Figure 2.2 — Percentage of Total using ALL

What Is Happening Here?

When DAX evaluates Overall Total Sales in a row filtered to Product = "Laptop":

1. CALCULATE re-evaluates SUM of Sales Amount.
2. ALL(Sales) removes the product filter — so the calculation sees ALL products.
3. The result is the grand total across all products.
4. DIVIDE then divides Laptop Sales by the grand total to give the percentage.

Sample Output — Product Sales Percentage

Product	Sales Amount (₹)	Overall Total (₹)	% of Total
Laptop	3,20,000	10,00,000	32.0%
Mobile Phone	2,80,000	10,00,000	28.0%
Tablet	1,50,000	10,00,000	15.0%
Accessories	2,50,000	10,00,000	25.0%
Total	10,00,000	10,00,000	100.0%

Figure 2.3 — Product Sales Percentage Table

2.3 The ALLEXCEPT Function

ALLEXCEPT(Table, Column1 [, Column2, ...])	Removes all filters from a table EXCEPT the specified columns. Those specified columns keep their filter context active.
---	--

How Is ALLEXCEPT Different from ALL?

ALL	ALLEXCEPT
Removes ALL filters from the table or column	Removes ALL filters EXCEPT the ones you specify
Use when you need the grand total across everything	Use when you want the total within a group (e.g., within a Region)
Example: What is the total sales across ALL products and regions?	Example: What is the total sales within this region, across all products?

Figure 2.5 — ALL vs. ALLEXCEPT

Syntax

```
DAX
ALLEXCEPT( <TableName>, <Column1> [, <Column2>, ...] )

-- Example:
ALLEXCEPT( Sales, Sales[Region] )
-- This removes all filters from Sales table EXCEPT the Region filter.
-- The Region filter is kept active.
```

Figure 2.6 — ALLEXCEPT Syntax

Example — Sales % Within Each Region

Scenario: You want a table showing each product's sales as a percentage of its region's total — NOT the overall total. This is where ALLEXCEPT is essential.

```
DAX
-- Region Total: Removes all filters EXCEPT Region
Region Total Sales =
CALCULATE(
    SUM( Sales[Sales Amount] ),
    ALLEXCEPT( Sales, Sales[Region] )
)
```

```
-- Percentage within Region:
Sales % Within Region =
DIVIDE(
    [Total Sales],
    [Region Total Sales],
    0
)
```

Figure 2.7 — Sales % Within Region using ALLEXCEPT

Sample Output

Region	Product	Sales (₹)	Region Total (₹)	% Within Region
North	Laptop	80,000	2,50,000	32.0%
North	Mobile	1,10,000	2,50,000	44.0%
North	Tablet	60,000	2,50,000	24.0%
South	Laptop	95,000	3,00,000	31.7%
South	Mobile	1,20,000	3,00,000	40.0%
South	Tablet	85,000	3,00,000	28.3%

Figure 2.8 — Sales % Within Region using ALLEXCEPT



Choosing Between ALL and ALLEXCEPT

Ask yourself: 'What context do I want to keep?'

If you want NO context (pure grand total) → use ALL

If you want to keep some grouping (e.g., Region) → use ALLEXCEPT and specify that column

If you only want to clear one specific filter → use ALL(Table[Column]) on just that column

Section 3: Variables in DAX — VAR and RETURN

As your DAX formulas grow more complex, readability and maintainability become critical. Variables (VAR) are one of the most impactful features in DAX — they make your formulas shorter, clearer, and faster.

3.1 What Is a Variable in DAX?

VAR (Variable)

A named storage location in a DAX formula that holds a value or expression. Variables are evaluated once and reused throughout the formula using the RETURN keyword.

Structure of a VAR Formula

DAX

```
VAR VariableName1 = <Expression1>
```

```
VAR VariableName2 = <Expression2>
```

```
RETURN
```

```
<Final Result using VariableName1 and/or VariableName2>
```

Figure 3.1 — VAR / RETURN Structure



Note

A DAX formula using VAR must always end with a RETURN statement.

You can define as many variables as you need.

Variable names cannot contain spaces — use camelCase or underscores.

Variables are local — they exist only within the formula they are defined in.

3.2 Why Use Variables?

Consider this formula without variables — calculating Year-over-Year Growth %:

DAX

```
-- Without Variables (harder to read and maintain):
```

```
YoY Growth % =
```

```
DIVIDE(
```

```
    SUM( Sales[Sales Amount] ) -
```

```
    CALCULATE( SUM( Sales[Sales Amount] ), SAMEPERIODLASTYEAR( 'Date'[Date] ) ),
```

```
    CALCULATE( SUM( Sales[Sales Amount] ), SAMEPERIODLASTYEAR( 'Date'[Date] ) ),
```

```
0
```

```
)
```

Figure 3.2 — YoY Growth % WITHOUT Variables

Now compare the same formula written with variables:

```
DAX
-- With Variables (clear, clean, and efficient):
YoY Growth % =
VAR CurrentYearSales = SUM( Sales[Sales Amount] )
VAR PreviousYearSales =
    CALCULATE(
        SUM( Sales[Sales Amount] ),
        SAMEPERIODLASTYEAR( 'Date'[Date] )
    )
VAR GrowthAmount = CurrentYearSales - PreviousYearSales
RETURN
    DIVIDE( GrowthAmount, PreviousYearSales, 0 )
```

Figure 3.3 — YoY Growth % WITH Variables

Without Variables	With Variables
CALCULATE(SUM ...) appears twice	CALCULATE(SUM ...) appears only once — stored in a variable
Hard to read — everything is nested	Easy to read — each step is clearly named
Slower — DAX re-evaluates the expression each time	Faster — the expression is evaluated once, then reused
Harder to debug — one long expression	Easy to debug — each variable can be tested separately

Figure 3.4 — With vs. Without Variables

3.3 Practical Examples Using Variables

Example 1 — Classifying Sales Performance

You want to classify each product's sales as 'High', 'Medium', or 'Low' based on thresholds. Variables make this logic clean:

```
DAX
Sales Category =
VAR TotalSales = SUM( Sales[Sales Amount] )
VAR HighThreshold = 200000
VAR LowThreshold = 80000
RETURN
```

```

IF( TotalSales >= HighThreshold, "High",
    IF( TotalSales >= LowThreshold, "Medium",
        "Low"
    )
)

```

Figure 3.5 — Sales Category using Variables

Example 2 — Profit Margin Calculation

```

DAX
Profit Margin % =
VAR TotalRevenue = SUM( Sales[Sales Amount] )
VAR TotalCost    = SUM( Sales[Cost Amount] )
VAR Profit       = TotalRevenue - TotalCost
RETURN
    DIVIDE( Profit, TotalRevenue, 0 )

```

Figure 3.6 — Profit Margin using Variables

Example 3 — Checking for Blank Values Before Dividing

A common issue in DAX is dividing by zero or a blank. Variables help you check the denominator before proceeding:

```

DAX
Safe Growth Rate =
VAR CurrentSales = SUM( Sales[Sales Amount] )
VAR PreviousSales =
    CALCULATE(
        SUM( Sales[Sales Amount] ),
        SAMEPERIODLASTYEAR( 'Date'[Date] )
    )
RETURN
    IF(
        ISBLANK( PreviousSales ) || PreviousSales = 0,
        BLANK(),
        DIVIDE( CurrentSales - PreviousSales, PreviousSales )
    )

```

Figure 3.7 — Blank-safe Growth Rate using Variables

Pro Tip

Use meaningful variable names — VAR PreviousYearSales is far better than VAR x.

Variables are evaluated in the row context that exists at the time VAR is defined.

You can use variables inside RETURN expressions, IF conditions, SWITCH, and any other DAX function.

Variables also make formulas faster because DAX only evaluates each VAR once, even if it is referenced multiple times in RETURN.



Section 4: Conditional Logic — IF and SWITCH

Conditional logic allows your DAX formulas to make decisions — returning different results based on different conditions. This is essential for creating dynamic classifications, labels, and scenarios in your reports.

4.1 The IF Function

IF

Evaluates a logical condition. Returns one value if the condition is TRUE, and a different value if the condition is FALSE.

Syntax

DAX

```
IF( <LogicalTest>, <ResultIfTrue>, [ResultIfFalse] )
```

-- LogicalTest : A condition that evaluates to TRUE or FALSE

-- ResultIfTrue : Value or expression returned when the condition is TRUE

-- ResultIfFalse: Value or expression returned when the condition is FALSE

(optional — returns BLANK() if omitted)

Figure 4.1 — IF Function Syntax

Example 1 — Flagging Orders Above Target

Scenario: Mark each order as 'Above Target' or 'Below Target' based on whether Sales Amount exceeds ₹1,00,000.

DAX

```
Performance Flag =
```

```
IF(
    SUM( Sales[Sales Amount] ) >= 100000,
    "Above Target",
    "Below Target"
)
```

Figure 4.2 — Simple IF for Performance Flag

Example 2 — Nested IF (Multiple Conditions)

Scenario: Classify each product's sales into three tiers.

DAX

```
Sales Tier =
```

```
VAR Sales = SUM( Sales[Sales Amount] )
```

```
RETURN
```

```
IF( Sales >= 300000, "Platinum",
```

```

IF( Sales >= 150000, "Gold",
  IF( Sales >= 75000, "Silver",
    "Bronze"
  )
)

```

Figure 4.3 — Nested IF for Sales Tier

⚠ Important

Avoid deeply nested IF statements — they become very hard to read and debug.

If you have more than 3–4 conditions, use SWITCH instead (covered in the next section).

Always use a VAR to store the value being tested — this avoids repeating SUM() in each branch.

4.2 The SWITCH Function

SWITCH

Evaluates an expression against a list of values and returns the result for the first matching value. Cleaner and more readable than multiple nested IFs.

Syntax — Two Forms

DAX

-- Form 1: Match against specific values

```

SWITCH(
  <Expression>,
  Value1, Result1,
  Value2, Result2,
  ...
  [DefaultResult]
)

```

-- Form 2: SWITCH(TRUE()) — for range-based conditions

```

SWITCH(
  TRUE(),
  Condition1, Result1,
  Condition2, Result2,
  ...
  [DefaultResult]
)

```

Figure 4.4 — SWITCH Syntax (both forms)

Example 1 — Region Labels Using SWITCH (Form 1)

Scenario: Convert region codes (N, S, E, W) into full names.

```
DAX
Region Full Name =
SWITCH(
    Sales[Region Code],
    "N", "North Region",
    "S", "South Region",
    "E", "East Region",
    "W", "West Region",
    "Unknown Region" -- Default if no match
)
```

Figure 4.5 — SWITCH for Region Labels

Example 2 — Sales Tier Using SWITCH(TRUE()) (Form 2)

Scenario: The same Sales Tier example from the IF section — but written more cleanly with SWITCH(TRUE()).

```
DAX
Sales Tier =
VAR Sales = SUM( Sales[Sales Amount] )
RETURN
SWITCH(
    TRUE(),
    Sales >= 300000, "Platinum",
    Sales >= 150000, "Gold",
    Sales >= 75000, "Silver",
    "Bronze" -- Default
)
```

Figure 4.6 — Sales Tier using SWITCH(TRUE())

Nested IF	SWITCH(TRUE())
Gets hard to read with more than 3 conditions	Stays readable even with 6–8 conditions
Must close every IF with parentheses	One function call — less syntax overhead
Harder to add or remove conditions	Easy to add/remove a condition line
Both produce the same result — it's about readability	Preferred for complex, multi-condition logic

Figure 4.7 — Nested IF vs. SWITCH(TRUE()) Comparison

Example 3 — Dynamic Measure Selector Using SWITCH

A powerful pattern: Let users select which measure to display using a slicer.

```
DAX
-- Assume a disconnected table 'Metric Selector' with values:
-- 1 = Total Sales, 2 = Profit, 3 = Units Sold
```

```
Selected Metric Value =  
VAR SelectedMetric = SELECTEDVALUE( 'Metric Selector'[Metric ID], 1 )  
RETURN  
SWITCH(  
    SelectedMetric,  
    1, SUM( Sales[Sales Amount] ),  
    2, SUM( Sales[Profit Amount] ),  
    3, SUM( Sales[Units Sold] ),  
    BLANK()  
)
```

Figure 4.8 — Dynamic Metric Selector using SWITCH

ETHANS
CLOUD & AI ACADEMY



Section 5: Ranking Functions — RANKX

Ranking is a common requirement in business reports. You often need to know which product sold the most, which salesperson performed best, or which region ranked last. RANKX is the primary function for this in DAX.

5.1 The RANKX Function

RANKX	Returns the rank of a value within a specified table, based on a given expression. It evaluates the expression for each row and assigns a rank position.
--------------	--

Syntax

```
DAX
RANKX(
    <Table>,
    <Expression>,
    [Value],
    [Order],
    [Ties]
)

-- Table    : The set of rows over which to rank (usually ALL of a column)
-- Expression : The value to rank by (e.g., SUM of Sales)
-- Value    : The value to determine rank for (usually omitted — uses current row)
-- Order    : ASC = 1 is lowest value; DESC = 1 is highest value (default DESC)
-- Ties     : How to handle equal values — SKIP or DENSE
```

Figure 5.1 — RANKX Syntax

Example 1 — Rank Products by Sales

Scenario: In a table visual showing products and their sales, add a column showing the rank of each product.

```
DAX
Product Sales Rank =
RANKX(
    ALL( Sales[Product] ), -- Rank across ALL products
    SUM( Sales[Sales Amount] ), -- Rank by total sales
    , -- Omit value (uses current row)
    DESC, -- Rank 1 = highest sales
    DENSE -- No gaps in ranking if tied
)
```

Figure 5.2 — Product Sales Rank Measure

Breaking Down the Formula

ALL(Sales[Product]) — tells RANKX to evaluate across all products, removing any product filter the visual applies. Without ALL, RANKX would only see one product at a time and rank them all as 1.

SUM(Sales[Sales Amount]) — the expression being ranked. DAX evaluates this for each product.

DESC — rank 1 = highest sales. Use ASC if you want rank 1 = lowest.

DENSE — if two products are tied, they get the same rank, and the next rank is the immediate next number (no gap). Use SKIP if you want standard sports-style ranking.

SKIP vs. DENSE — Tie Handling

Product	Sales (₹)	Rank (SKIP)	Rank (DENSE)
Laptop	3,20,000	1	1
Mobile Phone	2,80,000	2	2
Tablet	2,50,000	3	3
Accessories	2,50,000	3	3
Headphones	1,80,000	5 (not 4)	4 (no gap)

Figure 5.3 — SKIP vs. DENSE Tie Handling

Example 2 — Top N Filter Using RANKX

Scenario: Show only the Top 5 products in a visual.

DAX

-- Create the rank measure first:

Product Rank =

```
RANKX(
    ALL( Sales[Product] ),
    SUM( Sales[Sales Amount] ),
    , DESC, DENSE
)
```

-- Then in a visual, use a filter where [Product Rank] <= 5

Figure 5.4 — Top N Filter using RANKX

Example 3 — Rank Within a Group (e.g., Region)

Scenario: Rank products within each region separately, so each region has its own Rank 1 product.

DAX

```
Rank Within Region =  
RANKX(  
    ALLEXCEPT( Sales, Sales[Region] ), -- Rank within the current Region only  
    SUM( Sales[Sales Amount] ),  
    , DESC, DENSE  
)
```

Figure 5.6 — Rank Within Region using ALLEXCEPT

**Key Insight: ALLEXCEPT in RANKX**

Using ALLEXCEPT(Sales, Sales[Region]) inside RANKX keeps the Region filter active.

This means products are ranked only within the current region — not across all regions.

Each region will have its own Rank 1, Rank 2, etc.

ETHANS
CLOUD & AI ACADEMY



Section 6: Performance Basics in DAX

As data models grow in size and complexity, the speed at which DAX measures evaluate becomes increasingly important. A slow formula can make a Power BI report feel unresponsive and frustrate users. This section covers the foundational principles for writing DAX that performs well.

6.1 How DAX Evaluates Formulas — The Engine Overview

Before optimising, it helps to understand how DAX works internally. Power BI uses two internal engines:

Engine	Role	Speed
Storage Engine (SE)	Retrieves data from the compressed in-memory data model. Works in batches.	Very fast — directly reads columnar data
Formula Engine (FE)	Evaluates the DAX formula logic — handles context, iteration, and calculations.	Slower — handles complex logic

Figure 6.1 — Storage Engine vs. Formula Engine

The goal of DAX performance optimisation is to maximise the work done by the Storage Engine and minimise the work sent to the Formula Engine.

6.2 Measures vs. Calculated Columns — When to Use Which

One of the most impactful decisions you can make for performance is choosing correctly between a Measure and a Calculated Column.

Aspect	Measure	Calculated Column
When calculated	At query time — only when the visual refreshes	At data refresh time — stored in the model
Memory usage	No storage — calculated on demand	Stored in the model — uses memory
Filter awareness	Fully filter-aware — responds to slicers and filters	Not filter-aware — always calculates all rows
Best used for	Aggregations: SUM, AVERAGE, COUNT, KPIs	Row-level categorisation: labels, flags, lookups
Example	Total Sales, YTD Sales, Rank	Product Category Flag, Region Label, Age Group

Figure 6.2 — Measures vs. Calculated Columns

⚠ Important

Never use a Calculated Column when a Measure would do the job — calculated columns bloat the model.

If you find yourself putting a SUM, CALCULATE, or FILTER inside a calculated column, that logic almost certainly belongs in a measure.

6.3 Avoid Using FILTER When You Can Use ALL

A common beginner pattern is using FILTER to restrict rows inside CALCULATE. However, FILTER is an iterator — it scans every row in the table, which is slow for large datasets.

DAX

```
-- Slow: Using FILTER unnecessarily
North Region Sales =
CALCULATE(
    SUM( Sales[Sales Amount] ),
    FILTER( Sales, Sales[Region] = "North" )
)

-- Fast: Use a direct filter argument instead
North Region Sales =
CALCULATE(
    SUM( Sales[Sales Amount] ),
    Sales[Region] = "North" -- Direct filter, no row-by-row scan
)
```

Figure 6.3 — FILTER vs. Direct Filter in CALCULATE



When Is FILTER Actually Required?

Use FILTER when you need to reference a MEASURE inside the filter condition.

Example: FILTER(ALL('Product'), [Total Sales] > 100000)

Direct filter arguments (e.g., Sales[Region] = "North") only work with column values, not measures.

When filtering on column values, always prefer the direct method — it uses the Storage Engine.

6.4 Use Variables to Avoid Repeated Calculations

We covered variables in Section 3, but it is worth revisiting them here for their performance benefits. Every time you reference an expression inside a formula without a variable, DAX re-evaluates it.

DAX

```
-- Inefficient: SUM evaluated twice
Profit Ratio =
DIVIDE(
    SUM( Sales[Sales Amount] ) - SUM( Sales[Cost Amount] ),
    SUM( Sales[Sales Amount] ),
```

```

0
)

-- Efficient: SUM evaluated once using VAR
Profit Ratio =
VAR Revenue = SUM( Sales[Sales Amount] )
VAR Cost = SUM( Sales[Cost Amount] )
RETURN
    DIVIDE( Revenue - Cost, Revenue, 0 )

```

Figure 6.4 — Avoiding repeated calculations with VAR

6.5 Performance Best Practices — Quick Reference

Practice	Why It Helps
Use Measures, not Calculated Columns, for aggregations	Measures calculate on demand; columns bloat model size
Prefer direct filter arguments over FILTER()	Direct filters use the fast Storage Engine
Use VAR to store repeated expressions	Prevents redundant re-evaluation
Avoid using COUNTROWS(FILTER())	Use CALCULATE(COUNTROWS(), condition) instead
Minimise the use of nested CALCULATE	Each CALCULATE call creates a new filter context layer
Keep your Date table lean	Only add columns you use — each column takes memory
Avoid using circular dependencies	Circular references force serial evaluation and crash reports

Figure 6.5 — DAX Performance Best Practices



Pro Tip

Use the Performance Analyzer in Power BI Desktop (View tab → Performance Analyzer) to measure how long each visual and measure takes to load.

Look for measures with high 'DAX query' time — those are the ones to optimise.

Also consider using DAX Studio (a free external tool) for deeper formula analysis and query traces.

Section 7: Practice Exercise — End-to-End Application

This exercise brings together all the concepts from this chapter into a single, cohesive scenario. You will build a set of DAX measures for a fictional company called Meridian Electronics.

7.1 Business Scenario

Meridian Electronics is a mid-sized electronics retailer operating across four regions in India: North, South, East, and West. The company tracks monthly sales, product performance, and sales team effectiveness.

The management team has requested the following metrics for their executive dashboard:

- 5. Year-to-Date (YTD) Sales for the current fiscal year (April–March)
- 6. Month-to-Date (MTD) Sales to track daily progress
- 7. Sales compared to the same month last year
- 8. Year-over-Year Growth Percentage
- 9. Sales as a percentage of overall total (using ALL)
- 10. Product ranking by region (using RANKX and ALLEXCEPT)
- 11. A performance classification label for each product

7.2 Sample Data Model Structure

Table Name	Key Columns	Role in Model
Sales	Date, Product ID, Region, Sales Amount, Cost Amount, Units Sold	Fact table — contains all transactions
Product	Product ID, Product Name, Category, Sub-Category	Dimension table — product details
Region	Region Code, Region Name, Manager	Dimension table — regional info
Date	Date, Day, Month, Month Name, Quarter, Year, Fiscal Year, Fiscal Month	Date table — marked as Date Table

Figure 7.1 — Meridian Electronics Data Model

7.3 Building the Measures Step by Step

Step 1: Base Measure — Total Sales

```
DAX
Total Sales =
SUM( Sales[Sales Amount] )
```

Figure 7.3 — Base Total Sales Measure

Step 2: YTD Sales (Fiscal Year — April to March)

```
DAX
YTD Sales (FY) =
TOTALYTD(
    SUM( Sales[Sales Amount] ),
    'Date'[Date],
    "03-31"
)
```

Figure 7.4 — Fiscal YTD Sales Measure

Step 3: MTD Sales

```
DAX
MTD Sales =
TOTALMTD(
    SUM( Sales[Sales Amount] ),
    'Date'[Date]
)
```

Figure 7.5 — MTD Sales Measure

Step 4: Sales Last Year and YoY Growth

```
DAX
Sales Last Year =
CALCULATE(
    SUM( Sales[Sales Amount] ),
    SAMEPERIODLASTYEAR( 'Date'[Date] )
)

YoY Growth % =
VAR CurrentSales = SUM( Sales[Sales Amount] )
VAR LastYearSales = [Sales Last Year]
RETURN
    IF(
        ISBLANK( LastYearSales ) || LastYearSales = 0,
        BLANK(),
        DIVIDE( CurrentSales - LastYearSales, LastYearSales )
    )
```

Figure 7.6 — Sales Last Year and YoY Growth % Measures

Step 5: Sales % of Total (Using ALL)

```

DAX
Grand Total Sales =
CALCULATE(
    SUM( Sales[Sales Amount] ),
    ALL( Sales )
)

Sales % of Grand Total =
DIVIDE( [Total Sales], [Grand Total Sales], 0 )

```

Figure 7.7 — Sales % of Grand Total

Step 6: Product Rank Within Region

```

DAX
Product Rank in Region =
RANKX(
    ALLEXCEPT( Sales, Sales[Region] ),
    SUM( Sales[Sales Amount] ),
    , DESC, DENSE
)

```

Figure 7.8 — Product Rank Within Region

Step 7: Performance Classification Label

```

DAX
Performance Label =
VAR TotalSales = SUM( Sales[Sales Amount] )
RETURN
SWITCH(
    TRUE(),
    TotalSales >= 500000, "★ Top Performer",
    TotalSales >= 250000, "✓ On Track",
    TotalSales >= 100000, "↗ Developing",
    "⚠ Needs Attention"
)

```

Figure 7.9 — Performance Label using SWITCH

7.4 Final Dashboard Layout

The following screenshot placeholder represents the final Power BI dashboard that uses all the above measures:

7.5 Common Mistakes to Avoid

Mistake	Why It Is a Problem	Correct Approach
Forgetting to mark the Date table	Time Intelligence functions will return errors or incorrect results	Right-click Date table in Model view → Mark as Date Table
Using FILTER where a direct filter works	FILTER scans every row — slow for large tables	Use CALCULATE(SUM(...), Column = Value) directly
Not using VAR in complex formulas	DAX re-evaluates the same expression multiple times — inefficient	Store repeated expressions in VAR
Using ALL() without CALCULATE	ALL by itself returns a table — it has no effect on totals without CALCULATE	Wrap ALL inside CALCULATE to apply it as a filter modifier
Nesting too many IF statements	Formulas become unreadable and hard to debug	Replace nested IF with SWITCH(TRUE())

Figure 7.11 — Common DAX Mistakes and Corrections

ETHANS
CLOUD & AI ACADEMY



Chapter Summary

In this chapter, you have covered the most important advanced DAX concepts that form the foundation of professional business intelligence work. Here is a recap:

Topic	Key Functions / Concepts	Primary Use
Time Intelligence	TOTALYTD, TOTALMTD, SAMEPERIODLASTYEAR	Cumulative totals, period comparisons, year-over-year analysis
Filter Modifiers	ALL, ALLEXCEPT	Removing or selectively keeping filters for percentage-of-total and ranked calculations
Variables	VAR, RETURN	Cleaner, faster, and more readable formulas
Conditional Logic	IF, SWITCH, SWITCH(TRUE())	Classifying, labelling, and making dynamic decisions in measures
Ranking	RANKX, DENSE, SKIP	Ranking items globally or within a group
Performance	Measures vs. Columns, direct filters, VAR, FILTER	Writing efficient DAX that loads fast in reports

Figure Summary — Advanced DAX Topics at a Glance



What Comes Next?

In the next chapter, you will dive into DAX table functions — SUMMARIZE, ADDCOLUMNS, CROSSJOIN, and GENERATE.

You will also learn about the concept of virtual tables — a powerful pattern for creating intermediate results inside a single measure.

These techniques build directly on the VAR and CALCULATE knowledge you have built in this chapter.